Pre-Signed URLs

one permission, one object, a short time

Watch the 4-min video

In one line: a pre-signed URL is a normal URL plus a signature, computed locally by your server, that grants exactly one operation (method + object + expiry) to whoever holds it; storage re-checks the signature on every request. It's a bearer token, so keep the scope narrow and the expiry short.

1 Private bucket, real users

The bucket must stay private, yet one user needs to upload a video into it and another needs to download a report from it. Handing out cloud credentials is out of the question, and proxying the bytes through your API makes it a byte pump (lesson 4). A pre-signed URL lets the client talk to storage directly, with a permission your server chose and signed.

2 A URL with a signature

```
https://myapp-uploads.s3.eu-west-1.amazonaws.com/uploads/u42/7f3c.mp4
?X-Amz-Algorithm=AWS4-HMAC-SHA256                          ── signature version 4
&X-Amz-Credential=AKIA…/20261113/eu-west-1/s3/aws4_request  ── who signed + scope
&X-Amz-Date=20261113T101500Z                               ── signed at
&X-Amz-Expires=900                                         ── valid 15 min (max 7 days)
&X-Amz-SignedHeaders=content-type;host                     ── headers the client must send
&X-Amz-Signature=5d672d79c15b13162d9279b0855cfba6…         ── HMAC over all of the above + method
note: the HTTP method (PUT) is part of what is signed; using the URL for GET or DELETE fails
```

Figure 1. An S3 pre-signed URL (Signature Version 4). Everything that defines the permission is in the query string and covered by the signature.

Your server, which holds credentials, signs a specific request: *this method, on this object, with these headers, until this time*. Change any part (another key, another method, a later expiry) and the signature no longer matches. The URL also carries the signer's identity, and S3 checks that identity's permissions **at the time of the request**: if the signing role loses `s3:PutObject`, every URL it signed stops working.

3 Signing is local and cheap

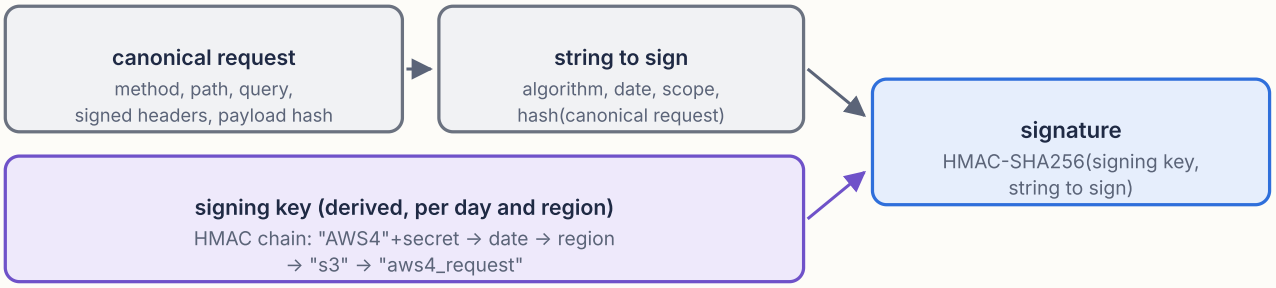


Figure 2. SigV4: the secret never leaves your server. Storage, which knows the same secret, recomputes the HMAC and compares.

Generating a URL makes **no network call**: the SDK computes an HMAC-SHA256 from the secret key, date, region, service, method, key, query and signed headers. That takes microseconds, so you can sign a fresh URL for every request and never cache them. Two practical details:

- **Lifetime**: SigV4 allows up to 7 days, but a URL signed with *temporary* credentials (an IAM role in Lambda, ECS or EKS) dies when those credentials expire, possibly within an hour, whatever `X-Amz-Expires` says.
- **Expiry is checked when the request starts**: an upload or download that begins before expiry completes even if it runs past it.

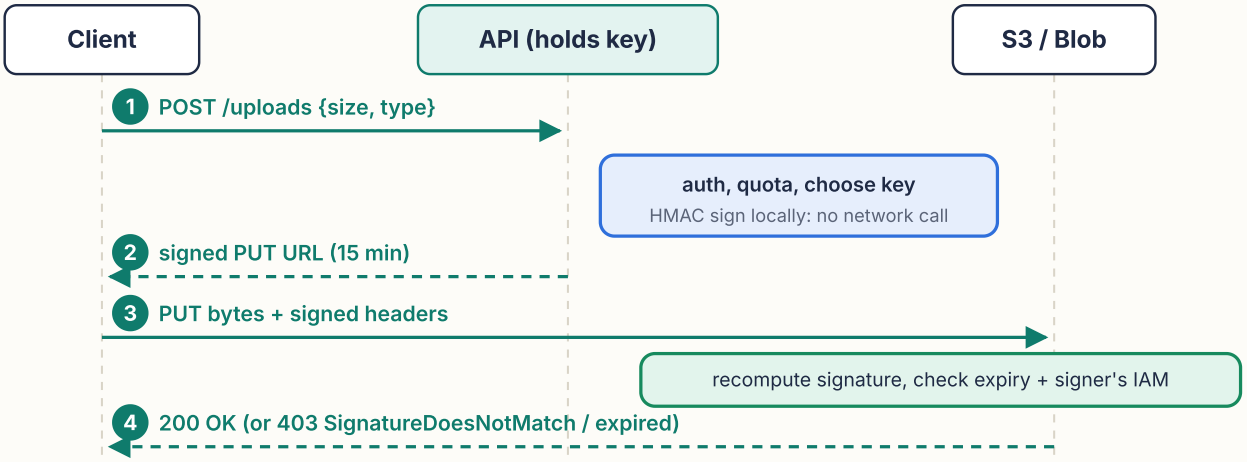


Figure 3. The API is in the control path only. Storage validates signature, expiry and the signer's current permissions on each request.

4 Azure SAS tokens and friends

Azure's equivalent is the **shared access signature** (SAS): query parameters such as `sp=cw` (permissions: create, write), `st` / `se` (start, expiry), `sip` (allowed IP range), `spr=https`, `sr=b` (one blob) and `sig`. Three kinds:

	Signed with	How to revoke early
S3 pre-signed URL	IAM user or role credentials (SigV4)	remove the signer's permission, deactivate its key, or a bucket-policy deny (e.g. on `s3:signatureAge` or source IP)
Azure service / account SAS	storage account key	ad hoc: only by rotating the account key (breaks every SAS); service SAS tied to a **stored access policy**: edit or delete the policy
Azure **user delegation SAS**	a user delegation key obtained with Microsoft Entra ID credentials (valid up to 7 days)	revoke the delegation keys or remove the principal's RBAC role; recommended by Microsoft
GCS signed URL (V4)	service account key or IAM signBlob	remove the service account's permission or its key; max 7 days

User delegation SAS is the safer default because no account key is involved; you can then disable shared-key authorization on the account entirely. Clock skew between machines can make a fresh SAS look "not yet valid": omit the start time or set it a few minutes in the past.

5 Scope is everything

- **One object, one method**: sign `PUT uploads/u42/7f3c.mp4`, never a container-wide or list permission for a client.
- **Server-chosen keys**: generate the key (UUID) on the server, so a client can't overwrite another user's file or pick a path like `../`.
- **Short expiry**: minutes for uploads to start, minutes to hours for downloads; days only for rare, deliberate sharing.
- **Constrain content**: a pre-signed PUT can't express a size range. Use a pre-signed **POST** policy with `content-length-range` and a `Content-Type` condition, or sign the exact `Content-Type` header. You can even sign a checksum header (`Content-MD5` or `x-amz-checksum-sha256`) declared at initiation, so only that exact file is accepted.
- **Least-privilege signer**: a dedicated role that can only `PutObject` on `uploads/*` limits damage if URLs or the role leak.
- **Downloads**: add `response-content-disposition=attachment; filename="report.pdf"` to the signed GET so the browser downloads with a friendly name instead of rendering the file.

```
post = s3.generate_presigned_post(
    Bucket="myapp-uploads", Key=f"uploads/{user_id}/{uuid4()}",
    Conditions=[["content-length-range", 1, 2_000_000_000],   # ≤ 2 GB
                ["starts-with", "$Content-Type", "video/"]],
    ExpiresIn=900)                                            # 15 minutes
# returns {url, fields}: the client POSTs the fields + file as multipart/form-data
```

Figure 4. boto3: a pre-signed POST that enforces size and type. The policy is part of what is signed, so the client can't change it.

6 The risks

A pre-signed URL is a **bearer token**: whoever holds it can use it. It leaks through access and proxy logs (query strings are logged), browser history, analytics tools that record URLs, and links pasted into chats or tickets. Most can't be revoked one by one, so short expiry is the main defence; add IP conditions

where clients have stable addresses (rarely true on mobile). Unique, short-lived URLs also defeat CDN caching: for content served to many users, use CDN signed URLs or cookies instead (lesson 11).

7 Example: a 2 GB video straight to S3

WORKED EXAMPLE: UPLOADING A 2 GB VIDEO

The app calls `POST /videos` with size 2,000,000,000 and type `video/mp4`. The API checks the user's remaining quota, creates a `videos` row (`pending`), picks the key `uploads/u42/7f3c.mp4` and signs a PUT URL that expires in **15 minutes** with `content-type: video/mp4` among the signed headers, using a role that may only write under `uploads/`. The client starts the upload at minute 1; it runs for 40 minutes on a slow link and still succeeds because expiry is checked at the start. S3 emits `ObjectCreated`, a consumer verifies the size and sets the row to `uploaded`, and the API never touched the 2 GB.

2 GB is under S3's 5 GB single-PUT limit, but one dropped connection restarts it from zero. In production you'd use multipart with a pre-signed URL per part (lesson 6).

TRADE-OFFS

- No credentials on clients and no bytes through your servers, but authorization happens once, at signing time.
- Short expiry limits leaks but can break slow or queued clients; re-sign on demand rather than lengthening expiry.
- Account-key SAS is simple but revocation means rotating the key for everyone; user delegation SAS is safer but needs Entra ID setup.
- Per-user URLs are uncacheable; CDN signing (lesson 11) suits widely read private content.

COMMON MISTAKES

- Signing with long-lived admin keys, or for a whole bucket or container.
- Letting the client choose the object key.
- 7-day URLs "to be safe", which then show up in logs and chat history.
- Logging full URLs (including the signature) in application or proxy logs.

8 Key terms

Pre-signed URL: URL carrying a signature that grants one operation until expiry

SAS token: Azure shared access signature: permissions, expiry, IP, signature

Stored access policy: server-side policy a service SAS can reference, editable to revoke

SigV4: AWS request-signing algorithm based on HMAC-SHA256

User delegation SAS: SAS signed with an Entra ID–derived key, not the account key

Bearer token: a credential that works for whoever holds it

9 Interview questions

Q1. How do pre-signed URLs work? BASIC

The server signs one permission (object, method, expiry, optional headers) with its key; anyone holding the URL can do exactly that without credentials, and storage recomputes and checks the signature.

Q2. What are their risks? BASIC

They're bearer tokens that can leak via logs, history or chats and usually can't be revoked individually; keep scope narrow, expiry short, and enforce size and content type.

Q3. A pre-signed download URL was posted publicly. How do you kill it before it expires? SCENARIO

Remove the signing identity's permission or rotate/deactivate its key (S3), or revoke the stored access policy / delegation keys (Azure); then move to shorter expiries and a dedicated signer.

Q4. How do you stop clients uploading 10 GB files with an upload URL? SCENARIO

Use a pre-signed POST policy with content-length-range (or a signed exact size/checksum), check quota before signing, and verify the real size after upload.

Q5. Your Lambda signs URLs with a 7-day expiry, but users report them failing after a few hours. Why? SCENARIO

The URL was signed with the function role's temporary credentials; it becomes invalid when those session credentials expire. Sign with a longer-lived identity if long URLs are really needed.

Q6. Does generating a pre-signed URL check that the object exists or that the caller is allowed? TRICKY

No; signing is purely local. Existence and permissions (of the signer) are checked by storage when the URL is used.

Q7. A 15-minute upload URL is used for an upload that takes an hour. Does it fail? TRICKY

No: expiry is checked when the request starts, so a transfer that begins in time completes. Multipart uploads need a valid URL per part, so later parts need fresh URLs.

Next: SD-O-006 · Resumable Uploads backendroadmap.com